Data Engineering [D.E]
Assignment-03

Name: A. Aswini
Branch:- CSD
Roll_no: 2SNUSA4401
2nd year

30/30

1) a) Summarize the conditions under which an organization might start with a monolithic, tightly coupled architecture versus when they should prioritize loose coupling.

Ans An organization may start with a monolithic tightly coupled architechecture when the application is small, less complex and requires fast development. In monolith, all application logic and the user interface exist in a single codebase and database, which makes development and deployment simpler for small teams.

Organizations should prioritize loose coupling through microservices architecture when systems become large and complex. Microservices divide the application into separate, decentralized services, each handling a specific function. This improves scalability, flexibility & reliability because if one service fails, the other can continue functioning independently.

b) Discuss in detail how loose coupled systems, making reversible decisions and prioritize security principles help in desinging a good data architecture.

Loose coupling, reversible decisions and prioritizing security are keyprinciples for designing good data architecture.

A loosely coupled System: divides a large System into small independent components that communicate through APIs or messaging systems. This allows teams to develop

update & deploy components independently without affecting other parts of the system. It improves scalability, flexibility & System maintainability.

Reversible decisions: refer to making architectural choices that can be easily changed later. In a rapidly changing technology landscape, choosing flexible & replaceable tools keep the architecture agile & adaptable.

Prioritizing Security: ensures that data and systems are protected from threats. Modern architecture follow the zero-trust model, where no user or system is automatically trusted, and every access request must be verified.

5) Together these principles help create a flexible, Scalable and secure data architecture.

2) what are the advantages and challenges of using APIs for data generation and exchange?

Advantages of using APIs for data generation & exchange

1. Standardized communication: APIs provide a standard method for different application to access & exchange data.
2. Loose coupling: Systems remain independent; changes in one system usually do not affect others.
3. Real-Time data access: APIs allow real-time & near real-time data transfer b/w applications.
4. Scalability: APIs enable organizations to easily scale services & integrate new applications.
5. Automation: Data can be automatically generated, collected and transfered b/w systems.
6. Easy integration: APIs allow integrate with external platforms such as cloud services & third party applications

challenges of using APIs for data generation & exchange:
Rate limits: APIs often restrict the number of requests within no specific time.
Security issues: APIs require proper authentication & authorization to protect data.
Versioning problems: APIs updates or changes may break existing integrations.
Latency & reliability: Data exchange depends on network speed & availability, which may cause delays.

3) Discuss in detail file system, OLTP, OLAP, CRUD, logs, APIs message queues, event stream platform & webhooks.

Ans: File Systems:
- A file system stores data as files in directories → Size → Append
  ex: CSV, JSON, text files, images.
- used for storing & exchanging large datasets. Commonly used in data lakes & batch processing.

2. OLTP (Online Transaction processing):
- Handles day-to-day operational transactions, supports large no. of small, fast transactions.
  ex: banking systems, online shopping orders.
- ensures data accuracy & consistency.

3. OLAP (Online analytical processing):
- used for data analysis & reporting, works with large volume of historical data.
- supports complex queries & business intelligence used in data warehouse & dashboards.

4. CRUD operations:-
CRUD represents basic database operations.
Create (insert new data), read (retrieve data), update (modify data), Delete (remove) data.

5. logs: logs record system & application events used for monitoring. ex:- Server logs, application logs, security logs and also used in troubleshooting

6. APIs [Application Programming interfaces]:
- APIs allow communication b/w software systems.
- used to exchange & integrate data b/w application
ex:- social media APIs, payment APIs.

7. Message Queues:-
- Message queues enable asynchronous communicate b/w services.
- Messages are stored in a queue & processed later.

8. Event streaming platforms
process continous real-time stream of data events.
used in applications like IOT, user activity tracking etc.,

9. Webhooks: ~~ex:~~
Webhooks are event-driven HTTP callbacks, A System sends data automatically when an event occurs.
ex. payment success notifications.

10 Messages are discrete & singular signals in an event-driven system.